

CSA0302 – Data Structures

Activity - 2

A velmurugan

192521074

Part A: Stack Implementation Using Array

Write a menu-driven C program to implement a stack using an array. Operations to Implement Push an element Pop an element Peek the top element Display stack elements Check whether the stack is empty Check whether the stack is full Exit Sample Menu STACK USING ARRAY

1. Push
2. Pop
3. Peek
4. Display
5. Check Empty
6. Check Full
7. Exit

Enter your choice: Conditions to Handle

- Display when the stack is full.
- Display when the stack is empty.
- Display the top element using the Peek operation.
- Display stack elements from top to bottom.

Program:

```
#include <stdio.h>
```

```
#define MAX 5
```

```
int stack[MAX];
```

```
int top = -1;
```

```
void push() {
```

```
    int item;
```

```
    if (top == MAX - 1) {
```

```
        printf("Stack is Full\n");
        return;
    }
    printf("Enter element: ");
    scanf("%d", &item);
    stack[++top] = item;
    printf("%d pushed into stack\n", item);
}
```

```
void pop() {
    if (top == -1) {
        printf("Stack is Empty\n");
        return;
    }
    printf("Popped element: %d\n", stack[top--]);
}
```

```
void peek() {
    if (top == -1) {
        printf("Stack is Empty\n");
        return;
    }
    printf("Top element: %d\n", stack[top]);
}
```

```
void display() {
    int i;
    if (top == -1) {
        printf("Stack is Empty\n");
        return;
    }
}
```

```
    }  
    printf("Stack elements (Top to Bottom):\n");  
    for (i = top; i >= 0; i--) {  
        printf("%d\n", stack[i]);  
    }  
}
```

```
void checkEmpty() {  
    if (top == -1)  
        printf("Stack is Empty\n");  
    else  
        printf("Stack is Not Empty\n");  
}
```

```
void checkFull() {  
    if (top == MAX - 1)  
        printf("Stack is Full\n");  
    else  
        printf("Stack is Not Full\n");  
}
```

```
int main() {  
    int choice;  
  
    while (1) {  
        printf("\nSTACK USING ARRAY\n");  
        printf("1. Push\n");  
        printf("2. Pop\n");  
        printf("3. Peek\n");  
        printf("4. Display\n");
```

```
printf("5. Check Empty\n");
printf("6. Check Full\n");
printf("7. Exit\n");

printf("Enter your choice: ");
scanf("%d", &choice);

switch (choice) {
    case 1: push(); break;
    case 2: pop(); break;
    case 3: peek(); break;
    case 4: display(); break;
    case 5: checkEmpty(); break;
    case 6: checkFull(); break;
    case 7: return 0;
    default: printf("Invalid Choice\n");
}
}
}
```

Output:

```
Output

STACK USING ARRAY
1. Push
2. Pop
3. Peek
4. Display
5. Check Empty
6. Check Full
7. Exit
Enter your choice: 1
Enter element: 10
10 pushed into stack

STACK USING ARRAY
1. Push
2. Pop
3. Peek
4. Display
5. Check Empty
6. Check Full
7. Exit
Enter your choice: 1
Enter element: 20
20 pushed into stack
```

```
Output

STACK USING ARRAY
1. Push
2. Pop
3. Peek
4. Display
5. Check Empty
6. Check Full
7. Exit
Enter your choice: 4
Stack elements (Top to Bottom):
20
10

STACK USING ARRAY
1. Push
2. Pop
3. Peek
4. Display
5. Check Empty
6. Check Full
7. Exit
Enter your choice: 7

=== Code Execution Successful ===
```

2. Part B: Stack Implementation Using Linked List

Write a menu-driven C program to implement a stack using a linked list. Operations to Implement Push an element Pop an element Peek the top element Display stack elements Check whether the stack is empty Exit Sample Menu

STACK USING LINKED LIST

1. Push
2. Pop
3. Peek
4. Display
5. Check Empty
6. Exit

Enter your choice:Conditions to Handle

- Create a new node dynamically during the Push operation.
- Delete the top node during the Pop operation.
- Display when the stack is empty.
- Display the elements from top to bottom.
- Handle memory allocation failure.

Program:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node {  
    int data;  
    struct Node *next;  
};
```

```
struct Node *top = NULL;
```

```
void push() {  
    struct Node *newNode;  
  
    newNode = (struct Node *)malloc(sizeof(struct Node));  
  
    if (newNode == NULL) {  
        printf("Memory Allocation Failed\n");  
        return;  
    }  
  
    printf("Enter element: ");  
    scanf("%d", &newNode->data);
```

```
newNode->next = top;
top = newNode;

printf("%d pushed into stack\n", newNode->data);
}
```

```
void pop() {
    struct Node *temp;

    if (top == NULL) {
        printf("Stack is Empty\n");
        return;
    }
```

```
    temp = top;
    printf("Popped element: %d\n", temp->data);
    top = top->next;
    free(temp);
}
```

```
void peek() {
    if (top == NULL) {
        printf("Stack is Empty\n");
        return;
    }
```

```
    printf("Top element: %d\n", top->data);
}
```

```

void display() {
    struct Node *temp;

    if (top == NULL) {
        printf("Stack is Empty\n");
        return;
    }

    printf("Stack elements (Top to Bottom):\n");

    temp = top;

    while (temp != NULL) {
        printf("%d\n", temp->data);
        temp = temp->next;
    }
}

void checkEmpty() {
    if (top == NULL)
        printf("Stack is Empty\n");
    else
        printf("Stack is Not Empty\n");
}

int main() {
    int choice;

    while (1) {
        printf("\nSTACK USING LINKED LIST\n");

```

```
printf("1. Push\n");  
printf("2. Pop\n");  
printf("3. Peek\n");  
printf("4. Display\n");  
printf("5. Check Empty\n");  
printf("6. Exit\n");
```

```
printf("Enter your choice: ");  
scanf("%d", &choice);
```

```
switch (choice) {  
    case 1: push(); break;  
    case 2: pop(); break;  
    case 3: peek(); break;  
    case 4: display(); break;  
    case 5: checkEmpty(); break;  
    case 6: return 0;  
    default: printf("Invalid Choice\n");  
}  
}  
}
```

Output:

Output	Output
<pre>STACK USING LINKED LIST 1. Push 2. Pop 3. Peek 4. Display 5. Check Empty 6. Exit Enter your choice: 1 Enter element: 15 15 pushed into stack STACK USING LINKED LIST 1. Push 2. Pop 3. Peek 4. Display 5. Check Empty 6. Exit Enter your choice: 1 Enter element: 30 30 pushed into stack</pre>	<pre>STACK USING LINKED LIST 1. Push 2. Pop 3. Peek 4. Display 5. Check Empty 6. Exit Enter your choice: 4 Stack elements (Top to Bottom): 30 15 STACK USING LINKED LIST 1. Push 2. Pop 3. Peek 4. Display 5. Check Empty 6. Exit Enter your choice: 3 Top element: 30</pre>

STACK USING LINKED LIST

1. Push
2. Pop
3. Peek
4. Display
5. Check Empty
6. Exit

Enter your choice: 2

Popped element: 30

STACK USING LINKED LIST

1. Push
2. Pop
3. Peek
4. Display
5. Check Empty
6. Exit

Enter your choice: 6

=== Code Execution Successful ===